

Clase 295 — OWASP Top 10 para aplicaciones con LLM

Parte: **15 — Seguridad de IA y machine learning** · Fuente: *OWASP Top 10 for Large Language Model Applications (OWASP Foundation)* 🕒 Duración estimada: **100 min** · Nivel: **Intermedio**

Objetivo

Dominar la taxonomía OWASP Top 10 para aplicaciones con LLM como marco de referencia para auditar y asegurar sistemas basados en modelos de lenguaje. El alumno aprenderá a identificar cada riesgo en una arquitectura real, mapearlo a controles y construir una checklist de auditoría reutilizable.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Enumerar** y explicar los 10 riesgos del OWASP Top 10 para LLM.
2. **Identificar** cada riesgo en un diagrama de arquitectura de una app con LLM.
3. **Asociar** cada riesgo con controles de mitigación concretos.
4. **Priorizar** riesgos según el contexto (chatbot, RAG, agente con herramientas).
5. **Construir** una checklist de auditoría OWASP-LLM aplicable a un proyecto.

Temas

#	Tema	Por qué importa
1	LLM01 Prompt Injection	El riesgo estrella; entrada no confiable como instrucción
2	LLM02 Insecure Output Handling	Salidas del LLM tratadas sin sanear (XSS, SSRF, RCE)
3	LLM03/04 Poisoning y DoS	Envenenamiento de datos y consumo desbordado de recursos
4	LLM05 Supply Chain	Modelos, plugins y datasets de terceros
5	LLM06 Sensitive Info Disclosure	Fuga de datos de entrenamiento o del contexto
6	LLM07/08 Plugins y Excessive Agency	Herramientas inseguras y permisos excesivos del agente
7	LLM09/10 Overreliance y Model Theft	Confiar ciegamente y robo del modelo

Definiciones y características

- **LLM01 – Prompt Injection:** entradas (directas o indirectas, vía documentos) que reprograman el comportamiento del modelo. *Característica:* la causa raíz es que datos e instrucciones comparten el

mismo canal.

- **LLM02 – Insecure Output Handling:** confiar en la salida del LLM sin validarla antes de renderizarla o ejecutarla. *Característica:* deriva en XSS, SSRF, path traversal o RCE aguas abajo.
- **LLM04 – Model Denial of Service:** entradas que fuerzan consumo excesivo (contextos enormes, bucles). *Característica:* impacta coste y disponibilidad.
- **LLM06 – Sensitive Information Disclosure:** el modelo revela secretos del prompt del sistema, datos de otros usuarios o del entrenamiento. *Característica:* agravado por memorización.
- **LLM08 – Excessive Agency:** el sistema concede al LLM demasiada capacidad de acción (permisos, herramientas, autonomía). *Característica:* un prompt malicioso se convierte en acciones reales.
- **LLM09 – Overreliance:** usuarios o sistemas confían en salidas sin verificar (alucinaciones). *Característica:* riesgo operacional y legal.
- **LLM10 – Model Theft:** exfiltración del modelo propietario (ver clase 294).

Nota: la lista OWASP para LLM se actualiza por versiones (2023, 2025); usa la más reciente y verifica los IDs vigentes.

Herramientas y preparación

- Documento oficial **OWASP Top 10 for LLM Applications** (última versión) y sus fichas por riesgo.
- Una app con LLM de ejemplo para auditar (chatbot de soporte con RAG, o un agente con herramientas).
- **garak** y **PyRIT** (clases 296–297) para pruebas prácticas de algunos riesgos.
- Plantilla de checklist en hoja de cálculo o Markdown.

Laboratorio guiado (auditoría aplicada)

Ejercicio de **análisis de arquitectura**, no de explotación.

1. **Elige una app real de referencia.** Ejemplo: "asistente de soporte que consulta una base de conocimiento (RAG) y puede crear tickets vía API".
2. **Dibuja el flujo de datos.** Usuario → prompt → recuperación de documentos → LLM → salida → acciones (crear ticket). Marca cada frontera de confianza.
3. **Recorre el Top 10 riesgo por riesgo.** Para cada uno, responde: ¿aplica a esta app? ¿dónde exactamente? ¿con qué impacto?
4. **Prompt injection (LLM01).** Identifica dónde entra texto no confiable (mensaje del usuario y documentos recuperados). Marca ambos como fuentes de injection directa e indirecta.
5. **Output handling (LLM02).** ¿La respuesta se renderiza en HTML? ¿Se pasa a otra herramienta? Define validación/escape necesarios.
6. **Excessive Agency (LLM08).** Lista las herramientas del agente y sus permisos. Aplica mínimo privilegio: ¿crear ticket necesita aprobación humana?
7. **Sensitive Disclosure (LLM06).** ¿Qué hay en el prompt del sistema? ¿Podría filtrarse? ¿Los documentos recuperados mezclan datos de distintos tenants?

8. **Construye la checklist.** Genera una tabla: Riesgo | ¿Aplica? | Evidencia | Control propuesto | Prioridad. Esta checklist será tu entregable reutilizable.

 Ejercicios

- 1. Para un chatbot público sin herramientas, prioriza los 3 riesgos más relevantes y justifica.
- 2. Da un ejemplo concreto de LLM02 que derive en XSS en un frontend.
- 3. Explica la diferencia entre prompt injection directa e indirecta con un caso.
- 4. Diseña un control de mínimo privilegio para un agente con acceso a correo.
- 5. Propón dos mitigaciones para LLM06 en una arquitectura multi-tenant.
- 6. Mapea tres riesgos del Top 10 a técnicas de MITRE ATLAS.

 Reto verificable

Entrega una **auditoría OWASP-LLM** de una aplicación (real o de ejemplo) con un diagrama de arquitectura anotado con fronteras de confianza y una checklist que cubra los 10 riesgos.

Criterio de aceptación: cada uno de los 10 riesgos aparece en la checklist con veredicto (aplica/no aplica) justificado, y para cada riesgo aplicable hay al menos un control concreto y una prioridad. Un tercero debe poder usar tu checklist para auditar otra app similar.

 Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Solo me preocupa el prompt injection"	Reduce la superficie a un riesgo. Recorre los 10; output handling y agency son igual de graves.
Confundir LLM01 y LLM02	Injection es la entrada; output handling es qué haces con la salida. Son controles distintos.
Ignorar la injection indirecta	Solo se filtra la entrada del usuario, no los documentos RAG. Trata todo contenido recuperado como no confiable.
Dar al agente todos los permisos	Excessive Agency. Aplica mínimo privilegio y human-in-the-loop para acciones sensibles.
Usar una versión desactualizada del Top 10	Los IDs cambian entre 2023 y 2025. Cita la versión y verifica los códigos.

 Preguntas frecuentes

 **¿El OWASP Top 10 para LLM reemplaza al Top 10 web clásico?** No. Lo complementa. Una app con LLM sigue siendo una app web con sus riesgos habituales; el Top 10 LLM añade los específicos del modelo.

 **¿Cuál es el riesgo más frecuente en la práctica?** Prompt injection (LLM01) y mal manejo de salida (LLM02) dominan los hallazgos reales, sobre todo cuando el LLM alimenta otras acciones o se renderiza sin escape.

 **¿Cómo priorizo si tengo recursos limitados?** Por impacto en tu arquitectura: si el LLM ejecuta acciones (agente), prioriza Excessive Agency e injection; si es solo conversacional, output handling y

disclosure.

? ¿Sirve para cumplimiento? Es un marco técnico, no una norma. Combínalo con NIST AI RMF e ISO/IEC 42001 (clase 300) para el marco de gobernanza y auditoría formal.

Referencias

- OWASP Top 10 for LLM Applications — <https://genai.owasp.org/llm-top-10/>
- OWASP LLM Applications Cybersecurity and Governance Checklist — <https://genai.owasp.org/>
- MITRE ATLAS — <https://atlas.mitre.org/>
- NIST AI 600-1, Generative AI Profile — <https://www.nist.gov/itl/ai-risk-management-framework>
- OWASP Foundation — <https://owasp.org/>

Siguiente clase

Clase 296 - Prompt injection y jailbreaks